Python Student Grade Tracker:

- **What does this program DO?** The user can add students and their grades, view all students with their averages, and see who passed or failed.
- **What does the user INTERACT with?** A simple text menu in the terminal — no buttons, no graphics, just input/output.
- **What concepts will you USE?**
  A. Functions → to organize each feature
  B. Loops → to keep the menu running
  C. Conditionals → to check pass/fail
  D. Try/Except → to handle bad input
  E. Variables & data types → everywhere

- **Description:** A **command-line application** that runs in your terminal. It allows a user to manage students and their grades in a simple, menu-driven interface. No fancy graphics, no buttons — just clean text input and output.
- **What problem does it solve?**
  A teacher or student wants a **quick and simple way** to record student grades and instantly know averages and pass/fail results — without using Excel or any external tool.

What are the boundaries of this project?

It WILL:

- Add students and multiple grades
- Calculate averages automatically
- Show pass or fail per student
- Handle wrong user input gracefully

Blueprint of the project:

Step 1 — What Does the Program Need to STORE?

Think about it simply. We have students, and each student has:

Student Name → text (string)

List of Grades → multiple numbers (we'll use a list - you'll learn this soon)

Average Grade → calculated from grades

Pass or Fail → determined by the average

Step 2 — What Can the USER DO? (Features)

1. Add a student + their grades
2. View all students + their average
3. Check who passed and who failed
4. Exit the program

Step 3 — How is the Program STRUCTURED? (Functions)

Each feature gets its own function. This is called **Separation of Concerns** — each function does ONE job only.

show_menu() → displays the menu options

add_student() → adds a student and their grades

view_students() → shows all students and averages

check_pass_fail() → shows who passed and who failed

main() → the engine that keeps everything running

Step 4 — How Does the Program FLOW?

```
START
  │
  ▼
main() starts
  │
  ▼
┌──────────────────────────────┐
│      show_menu()             │
│  1. Add Student              │
│  2. View All Students        │
│  3. Check Pass / Fail        │
│  4. Exit                     │
└──────────────────────────────┘
  │
  ▼
User enters choice (1, 2, 3, or 4)
  │
  ├── 1 ──▶ add_student()
  │         ├── Ask for name
  │         ├── Ask for grades (loop until 'done')
  │         ├── Try/Except → handle bad input
  │         └── Save to students dictionary
```

```
  │
  ├── 2 ──▶ view_students()
  │         ├── Check if dictionary is empty
  │         ├── Loop through students
  │         ├── Call calculate_average()
  │         └── Print name + average
  │
  ├── 3 ──▶ check_pass_fail()
  │         ├── Check if dictionary is empty
  │         ├── Loop through students
  │         ├── Call calculate_average()
  │         └── Print name + PASS or FAIL
  │
  └── 4 ──▶ Print goodbye message
        └── EXIT program
  │
  ▼
Loop back to show_menu()
(repeats until user picks 4)
  │
  ▼
END
```

Functions:

1. show_menu()

```
├── Input   : Nothing
├── Process : Prints the menu options
└── Output  : Menu displayed on screen
```

2. add_student()

```
├── Input   : Student name + grades from user
├── Process : Validates input, stores in dictionary
└── Output  : Confirmation message
```

3. view_students()

```
├── Input   : Nothing
├── Process : Loops through dictionary, calculates average
└── Output  : Prints each student + their average
```

4. check_pass_fail()

```
├── Input   : Nothing
├── Process : Loops through dictionary, calculates average
└── Output  : Prints each student + PASS or FAIL
```

5. calculate_average()

```
├── Input   : A list of grades
├── Process : Adds all grades, divides by count
└── Output  : Returns the average number
```

6. main()

```
├── Input   : User menu choice
├── Process : Keeps program running, directs to right function
└── Output  : Calls the correct function based on choice
```

Error Handling Plan:

Situation	How We Handle It
User enters a letter for choice	→ Try/Except → "Invalid input"
User enters a letter for grade	→ Try/Except → "Please enter a number"
User enters a negative grade	→ Conditional → "Grade cannot be negative"
User enters grade over 100	→ Conditional → "Grade cannot exceed 100"
No students added yet	→ Conditional → "No students found"
Student name already exists	→ Conditional → "Student already exists"

📁 File Structure

```
student_grade_tracker.py
│
├── students = {}              ← global data store
│
├── def calculate_average()    ← helper function
├── def show_menu()            ← display menu
```

```
├── def add_student()          ← feature 1
├── def view_students()        ← feature 2
├── def check_pass_fail()      ← feature 3
│
└── def main()                 ← engine / entry point
      └── called at the
          bottom of the file
```

📐 Build Order

```
Step 1 → show_menu()         ← simplest, builds confidence
Step 2 → main()              ← the engine, menu loop
Step 3 → add_student()       ← core feature, teaches dictionary + list
Step 4 → calculate_average()  ← small helper, teaches logic
Step 5 → view_students()     ← uses calculate_average()
Step 6 → check_pass_fail()   ← uses calculate_average()
Step 7 → Error Handling      ← polish everything
```

- **One Thing You Will Learn NEW in This Project**
  Your background doesn't include **lists** or **dictionaries** yet. You will need them to store students and grades. Don't worry — I will teach you **exactly what you need when you need it**, right before you use it. Nothing extra.